biohub-cell-tracking-during-development

10位解法: Grandmaster が駆動するエージェント型アプローチ

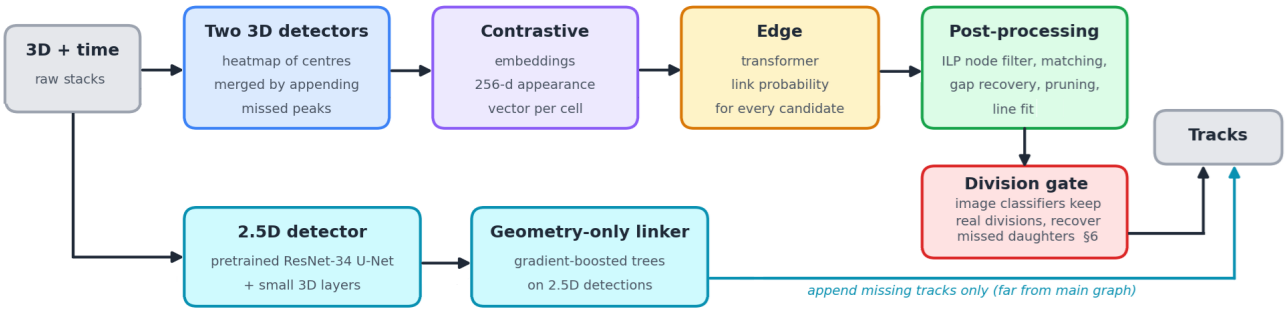
Rank	10
Team	Kaggle Agent
Author	Theo Viel
Topic ID	744980
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/biohub-cell-tracking-during-development/discussion/744980
Retrieved with Kaggle CLI on 2026-10-04.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Biohub と Kaggle に、このコンペティションを開催していただいたことを感謝します。私たちの解法はモジュール型のパイプラインです。2つの 3D 検出器がノードの主集合を提供し、contrastive モデルが各ノードに外観埋め込みを与え、transformer が候補リンクをスコアリングし、後処理チェーンがトラックを構築し、小さな画像分類器のアンサンブルが division を整理・回復します。最後に、別の 2.5D 検出器が取りこぼしたトラックを追加します。

作業は Claude、Codex、そして私たちの Kaggle Agent の間で分担し、そのすべてを私たちの Kaggle 経験で舵取りしました。write-up はできるだけ簡潔にまとめるよう努めました。不明な点があれば、遠慮なくコメントで質問してください。



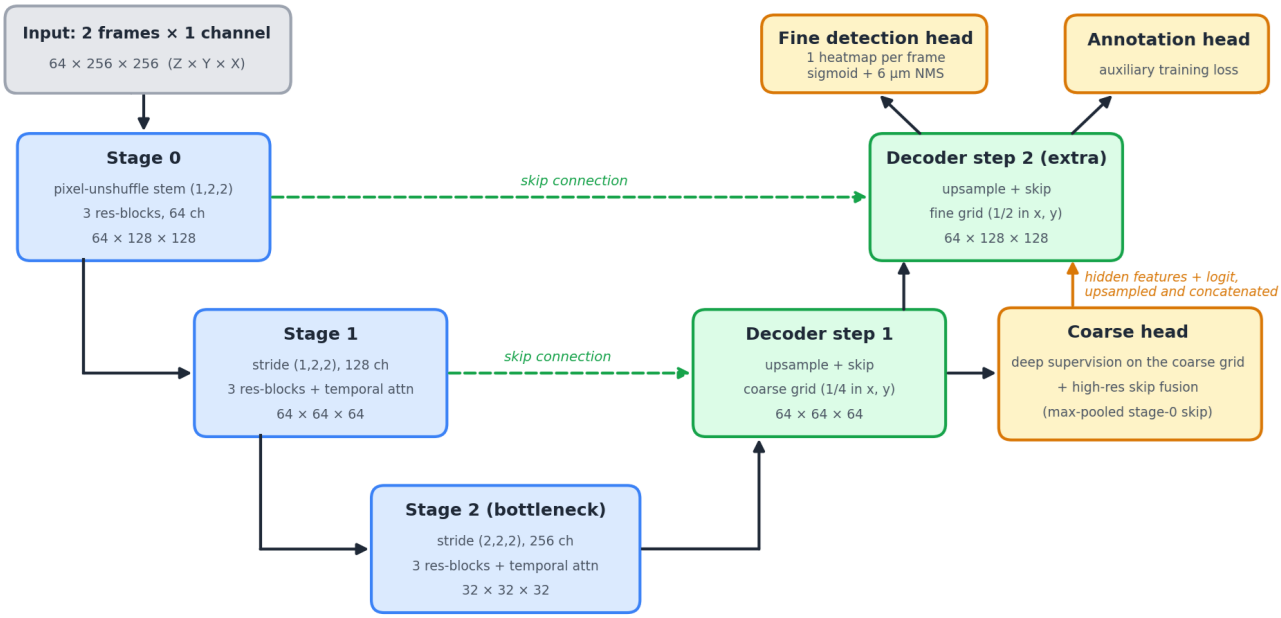
結果

6bba	44b6	CV	Public LB	Private LB
0.8824	0.9241	**0.903**	0.937 (1838th)	**0.948 (10th)**

public セットと private セットは2つの別々の胚であり、これはプロービングで容易に確認できます。この情報を踏まえると、モデルを学習・検証する正しい方法は、2 fold の leave-one-embryo-out の設定を用いることです。報告している値は2つの胚に対するマクロ平均です。提出には、両方の学習用胚で学習した別の full-fit モデルを使いました。

私たちの public の順位は上位にはほど遠いものでしたが、公開されているモデルをベースにしたチームは private で順位を落とすだろうと予想していました。未解決の問題は、私たちのアプローチが起こりうるドメインシフトに耐えられるかどうかでした。結果、耐えられました :)

1a. 3D 検出器



アーキテクチャ

1つ目のモデルは、2フレームのウィンドウにわたる temporal attention を備えた 3D U-Net で、公開ノートブックのものと似ていますが、以下の変更を加えています。

- **フル解像度の入力。** z 軸は x や y よりもずっと粗いため、最初の2ステージは (1, 2, 2) でのみダウンスケールし、グリッドを等方的に近づけます。
- **Unshuffle stem。** 最初のダウンサンプルでは、ストライドやプーリングの代わりに x/y ピクセルの 2×2 ブロックをそれぞれチャネルに折りたたみ(pixel-unshuffle)、細かな x/y の情報を捨てずに保持します。
- **高解像度スキップ融合。** ヘッドのグリッドより細かいスキップ接続もヘッドに入力し、検出マップが最も細かい特徴の恩恵を受けられるようにします。
- **deep supervision 付きの追加デコードステップ。** ノードグリッドの上にさらに細かい検出マップを生成し、より粗いヘッドがそれを教師付けします。
- **より大きなモデル。** ステージごとに 3 ブロック、特徴数は2倍です。

図には推論時のボリュームサイズを示しています。学習時はより小さな空間クロップを使います。

学習戦略

- **データ。** (32 x 128 x 128) のクロップで強いデータ拡張をかけて学習します。最初は合成データで、次に実データと合成データの混合で学習します。
- **ターゲットと損失。** ターゲットは細胞中心の周りの小さなガウシアンブロブで、モデルはこれを focal loss で学習します。
- **スパースなアノテーションに対する擬似ラベル。** アノテーションされていない細胞は背景ではありません。擬似ラベリングを 3 ラウンド行いました。前のモデルの検出を低重みの正例として追加し、実在する細胞の近くでは負例から除外します。
- **検出器の上に小さな transformer を接続。** これは U-Net を凍結した状態で学習し、位置情報のみを使い、コンペティションの評価指標を用いた早期の validation スコアを得るためだけに使います。本物のリンカーは後で、追加の細胞特徴を用いて学習します。

デコード

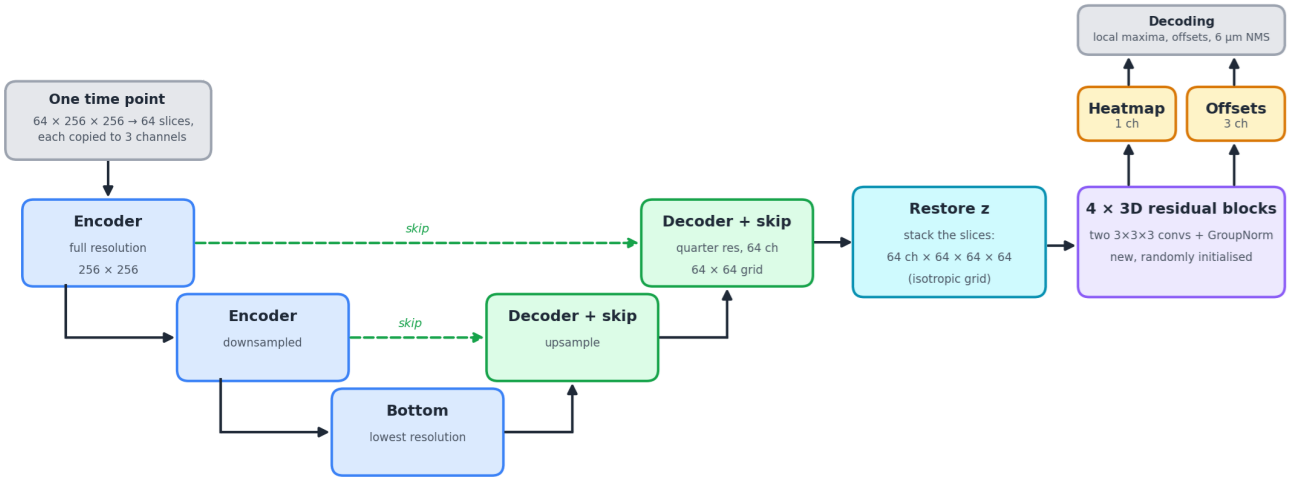
max-pooling ではなく、固定された物理半径(6 µm)での貪欲な non-maximum suppression を用います。これにより、どのストライドでも半径が同じ意味を持ちます。検出閾値は1つ目のモデルで 0.45、2つ目のモデルで 0.40 を使っています。

ノード集合の統合

1つ目の 3D モデルの検出をベースとして保持し、2つ目のモデルの検出のうち、同じフレーム内で 7 µm 以内にベースの検出が存在しないものだけを追加します。2つ目の検出器は隙間を埋めるだけで、ベースの検出を移動したり置き換えたりすることはありません。これには2つの validation 胚で閾値を調整する必要があり、それが非公開の胚で最適であるという保証はありません。

この方法は、エンドツーエンドのパイプラインにおいて、logit の平均化や、2つのノード集合を平均してコンセンサスの重心にする方法よりも最終的に良い結果となりました。

1b. 2.5D 検出器(取りこぼしたトラックのための後段融合)



アーキテクチャ

スパースにアノテーションされた胚が2つしかない状況では、3D 検出器をスクラッチから学習すると、モデルが学ぶべきことが多く残ります。そこで代わりに、2018 Data Science Bowl で核のセグメンテーション用に事前学習された ResNet-34 U-Net を再利用し、その 2D 特徴の上に小さな 3D 拡張を学習させます。

このモデルは一度に1つの時点を、64×256×256 ボクセルのボリュームとして見ます。各 z スライスを3チャネルに複製し、同じ事前学習済み 2D ネットワークに通します。エンコーダとデコーダは、エンコーダのスキップ融合を含めて 1/4 解像度の特徴マップまでを保持します。各スライスは 64×64 グリッド上で 64 チャネルを生成します。

次に、z を空間次元に戻します。元のボクセル間隔は z 方向で 1.625 µm、x/y 方向で 0.40625 µm なので、この 1/4 解像度のグリッドは等方的になります。新たに初期化した 4 つの 3D 残差ブロックがスライス間で情報を混ぜ合わせます。各ブロックは GroupNorm 付きの 3×3×3 畳み込みを2つ持ちます。2つの 1×1×1 ヘッドが、細胞中心の heatmap と3つの座標オフセットを予測します。

学習戦略

- **スパースな教師信号。** 3D モデルの focal loss ではなく、ソフトなガウシアンターゲットと重み付き BCE 損失を使います。公式にアノテーションされた中心の周りでは Smooth L1 損失でオフセットを教師付けします。
- **擬似ラベル。** 3D 検出器による擬似ラベルを弱い検出の正例として使い、補助損失の重みは 0.2 とします。
- **ファインチューニング。** まずデコーダを含む事前学習済み 2D トランク全体を凍結し、新しい 3D 層とヘッドを学習します。その後、BatchNorm の統計量は凍結したまま、10 倍低い学習率でトランクの凍結を解除します。

デコード

heatmap の 3×3×3 の局所最大値を取り、座標オフセットを適用し、6 µm での貪欲な物理 NMS を実行します。したがって、2つの 3D モデルとは異なり、この検出器には物理 NMS の前に局所最大値のステップがあります。3つの 2.5D 検出器のアンサンブルも検討しましたが、選択した提出では単一のメインモデルのみを使っています。そのノードはメインの contrastive モデルや edge transformer には入りません。別個にリンクされ、セクション 6 で取りこぼしたトラックとして追加されるだけです。

2. Contrastive な外観埋め込み

公開カーネルでは、細胞の幾何情報をノード transformer に入力するために特徴マップを使っていました。しかし U-Net は細胞を背景から区別するよう学習されており、細胞同士を区別するようには学習されていないため、リンクにとって必ずしも最適な表現ではありません。この目的により適しているのは contrastive learning です。3D U-Net を contrastive loss でファインチューニングするなど、いくつかの方法を試しましたが、最終的には独立した 2D モデルを使うのが最も良い結果でした。Heng がディスカッションで共有した ASCENT(link)の戦略を採用しています。

検出された各細胞は、256 次元の L2 正規化された埋め込みを持ちます。モデルは、細胞の周りの小さなパッチ(3 つの z スライスをチャネルとし、48×48 ピクセル、約 20 µm)に対する DINO ViT-B/16 バックボーンと、その後に続く、同じフレームのすべての細胞に対して attention をとる 4 層の transformer から成ります。モデルには座標は一切与えません。

このモデルは、GT のリンクを使わず、私たち自身の検出器の検出結果に対して完全に自己教師ありで学習します。各フレームを2回拡張し(位置のジッター、回転、弾性変形、スケーリング、輝度変化)、モデルは一方のビューのすべての細胞を、もう一方のビューの同じ細胞にマッチさせなければなりません。そのフレームの他のすべての細胞が負例となります(NT-Xent loss)。

3. Edge transformer

リンカーは HOCT 風の transformer で、統合されたノード集合を入力とします。

各ノードについて、次フレームの最近傍への候補リンクを生成します(各方向に 5 個、20 µm 以内)。transformer はまずノードに対して self-attention を実行し、次に候補 edge に対して実行し(これが「higher-order」の部分です)、候補ごとに1つのリンク確率を出力します。入力は以下のとおりです。

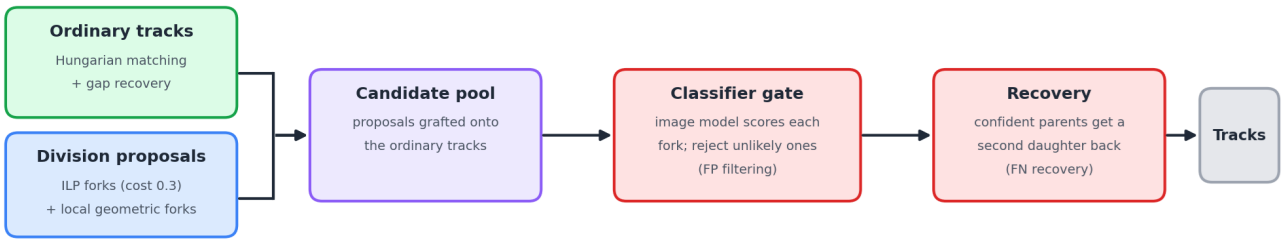
- 細胞の位置。胚の動きが良いリンクにペナルティを与えないようにグローバルなドリフト補正を施した後のもので、rotary なマルチスケール位置エンコーディングを使います。
- contrastive 埋め込み

- 検出器の確信度
- edge 特徴: 距離、埋め込みのコサイン類似度、そして各候補が、その両端の2つの細胞の他の候補の中で(距離および外観で)何位に位置するか。

4. 後処理

1. **ILP ノードフィルタ。** edge transformer が見つけた連続フレーム間の候補($p > 0.4$)に対して、グローバルなリンク ILP を解きます。最終的にリンクされた検出のみを保持し、edge は捨てます。
2. **割り当て。** リンクの確率に対する Hungarian マッチングで、フレームペアごとにリンクを選び直します。トラックごとのモメンタム項を加え、トラックの方向を急に変えるリンクにペナルティを与えます。
3. **ギャップの回復。** 途中で途切れ、数フレーム後に再開するトラックについて、動きがもっともらしく、かつその位置にまだ検出がない場合は、補間したノードで橋渡しし、その後割り当てを再実行します。
4. **枝刈り。** 5 ノードより短いトラックは、division の検証の前後で削除します。別系統の補助トラックのブランチは、独自の最小長 3 を使います。
5. **直線フィッティング。** 各ノードを、そのノード自身のトラックに沿ってフィットした直線に向けて移動させ、座標のばらつきを平滑化します。

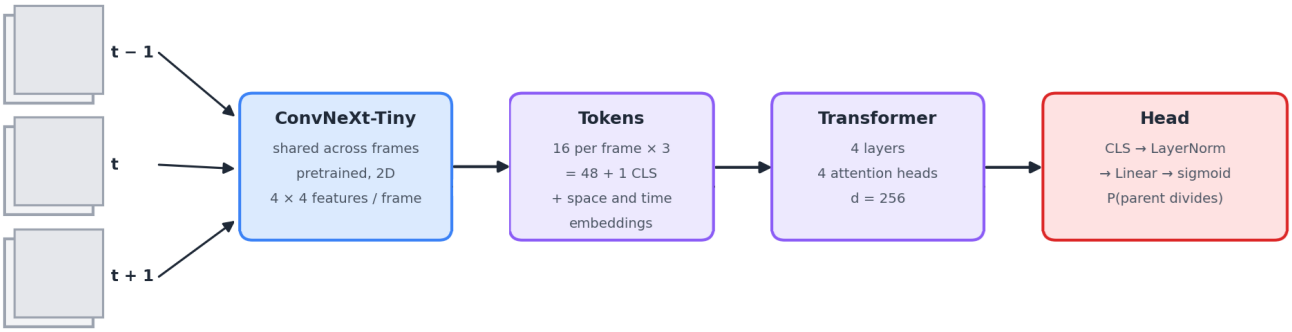
5. Division 分類器によるゲートと回復



候補の提案

division コストを下げた2回目の求解でリンク ILP 自身に分岐を提案させ、その追加の子リンクを、割り当てと回復を経たトラックに接ぎ木します。また、局所的な幾何ベースの提案器が、子を1つしか持たない親に、次フレームのまだ割り当てられていない2つ目の子を与えることもあります。これらの提案だけでは誤った division が多く生じるため、分類器がそれらを却下するのに必要な画像上の根拠を提供します。

分類器



純粋な画像モデルです。提案された各 division について、親を中心とした 3 フレームにわたる 32×32 のクロップを見ます。各フレームは隣接する 3 つの z スライスをチャネルとします。事前学習済みの ConvNeXt-Tiny が各フレームをエンコードし、小さな transformer が 3 フレームの空間トークンに対して実行され、親が分裂しているかどうかを予測します。

学習と利用

- **公式の division ラベル。** 正例はアノテーションされた分裂する親、負例はアノテーションされた直線的な継続です。これらを正例 40% にリバランスし、binary cross-entropy で学習します。
- **検出器の中心の周りで学習。** 学習クロップの半分はアノテーションを中心に、残りは 3D または 2.5D 検出器のマッチした検出を中心にします。これにより、division ラベルを変えずに、検出器の位置特定誤差にモデルをさらします。
- **アンサンブルと検証。** 独立に学習した 4 つのシードを平均し、平均確率が高い場合にのみ提案された division を保持します。そうでない場合は、確信度の低い方の子 edge を削除し、強い方の接続を残します。
- **再枝刈り。** ゲートの後に再び短いトラックを枝刈りします。今は削除された分岐のためだけに存在していた断片が、そうしなければ誤った edge とともに残ってしまうからです。
- **見逃した娘細胞の回復。** 分類器が親の分裂を非常に強く確信しているのに子が1つしかない場合、既存の検出の中から、より緩いリンク確率で、まだ割り当てられていない2つ目の娘細胞を探します。これは新しい細胞ではなく、既存の検出間のリンクを追加するものです。

最終的な division の Jaccard は 44b6 で 0.4444、6bba で 0.3117、すなわち **マクロ平均 0.3781** です。

6. 取りこぼしたトラックの追加

検出器の recall を上げても、エンドツーエンドで自動的に良くなるわけではありませんでした。近くにノードが追加されるとリンカーの選択肢が増え、その一部が誤ったリンクになったのです。そのため、2.5D 検出器は控えめに使っています。それ単独でリンクし、完成したメイングラフの隙間を埋めるだけで、既存のノードやリンクは変更しません。

- **幾何情報のみのリンカー。** 小さな勾配ブースティング木が、距離、近傍順位、検出器の確信度を使って、2.5D の検出間の候補リンクをスコアリングします。contrastive 埋め込みも edge transformer も使いません。
- **確信度の高い新しいチェーンのみ。** すべてのリンクの確信度が高く、ノードがメイングラフ内の既存のものから十分離れている、少なくとも 3 フレームの直線的なチェーンを追加します。それより短い断片は破棄します。
- **橋渡し。** 追加したチェーンは、既存のリンクを上書きすることなく、メイングラフの空いているトラック端点に接続することもできます。

ただし、CV 上の改善は小さいものでした(+0.002)。

お読みいただきありがとうございました!